

A From-Scratch Distributed MapReduce Engine

*Design, Fault Tolerance, Performance Analysis
and a Kafka Streams Comparison on Common Crawl*

SLR207, Distributed Computing

Télécom Paris

Final Written Report

Team

Camil Daou · Aymen Kabil · Rémi Deloye · Sébastien Riviere

Course	SLR207, Systèmes répartis / Distributed Computing
Subject	Build a MapReduce engine on 100 lab machines
Data set	Common Crawl (WET extracts)
Reference	Dean & Ghemawat, <i>MapReduce</i> , OSDI'04
Deliverables	engine, fault tolerance, 4 analyses, Kafka, this report

A self-built, dependency-free engine running over SSH on the
Télécom Paris cluster, no Hadoop, no HDFS, no root access.

July 1, 2026

Contents

1	Introduction and project context	3
1.1	What was asked, session by session	3
1.2	Design philosophy	3
1.3	Why Python	3
1.4	Repository map	4
1.5	Contributions and collaboration	4
2	Warm-up workflow: the CPU-load cluster (Day 1)	4
3	System architecture	5
3.1	Components	5
3.2	Storage model: NFS vs local disk	6
4	Network protocol design	6
4.1	Transport and message set	6
4.2	V1: the KISS protocol (no fault tolerance)	6
4.3	Reducer key distribution	7
4.4	Bootstrap	7
5	Common Crawl ingestion	7
6	The four analyses (pluggable jobs)	8
6.1	Observed results on a real Common Crawl split	8
7	The <code>bench.py</code> measurement library	10
7.1	Why we built it (and why not a bare chronometer)	10
7.2	How it works	10
7.3	How it was used, and how it helped	11
8	Optimisations	11
8.1	Baseline (naïve worker)	12
8.2	Optimisation 1: combine + bytes-regex + vectorised CRC32	12
8.3	Optimisation 2: parallel SSH shuffle with connection multiplexing	12
8.4	What the optimisations moved — and what they did not	13
8.5	Optimisation 3: on-demand ingestion — and its superlinear scaling	14
9	Performance instrumentation	15
10	Amdahl’s law: empirical validation	16
10.1	Method and the correctness rule	16
10.2	Cluster results (real machines over SSH)	16
10.3	Single-machine sweep (reproducible by one person)	17
10.4	Interpretation	18
11	Fault tolerance	18
11.1	1. Failure detection (heartbeat + lease)	18
11.2	2. Task reassignment (re-execution rules)	18
11.3	3. Atomic output commit (exactly-once)	19
11.4	4. Straggler backup tasks (§3.6)	19
11.5	5. Master recovery (designed, not implemented)	19
11.6	Demonstrating it	20

12 Validation	20
13 Kafka Streams comparison	20
14 Comparison: our system vs Hadoop vs Kafka Streams	21
15 Pain points: what broke and why	21
16 Deployment and reproducibility	22
16.1 Cluster deployment	22
16.2 One-command single-machine reproduction	22
16.3 Kafka demo	22
17 Implementation map: concept to code	22
18 Anticipated questions and design rationale (defense Q&A)	23
18.1 Memory: why we never saturated RAM, and how we would guarantee it	23
18.2 Storage and shuffle	25
18.3 Correctness and output	25
18.4 Fault tolerance	26
18.5 Performance and scaling	26
18.6 Deployment and ecosystem	26
19 Conclusion	27

1 Introduction and project context

This report documents, end to end, a distributed data-processing system built from scratch over five lab sessions for the SLR207 course at Télécom Paris. The goal was not to *use* a big-data framework but to *re-implement* the core ideas of Google’s MapReduce paper [1] on a real, shared, constrained cluster (the school’s 100+ lab machines), and to push it until it broke, then make it robust.

1.1 What was asked, session by session

The project was specified incrementally across five guideline sessions. Every single objective below is addressed in this report and implemented in the code.

Session	Objectives
Day 1	Infrastructure: pick 100 live machines from the school API, deploy a TCP load-average server to all of them over SSH/NFS, query them in parallel from a client, write a clean-up script. (§2)
Day 2	Study the MapReduce paper; design the Main/Workers protocol (KISS, no fault tolerance); space-time diagram; reducer key distribution $\text{hash}(k) \bmod R$; understand NFS vs local disk. (§3, §4)
Day 3	Formalise the protocol on sequencediagram.org; solve bootstrap and remote-read; run word-frequency on real Common Crawl splits; (advanced) design fault tolerance and compare with Kafka Streams. (§4, §5)
Day 4	Per-phase timing and bottleneck detection; remove the internal NFS by reading Common Crawl directly from Amazon; test at micro / real / large scale; prove Amdahl’s law empirically with a same-dataset speedup curve. (§9, §10)
Day 5	Push to breaking point and add fault tolerance; deploy Kafka Streams without Docker/root; implement three analyses beyond word count; compare our system vs Hadoop vs Kafka Streams (batch vs stream); write this report and present. (§11, §6, §13, §14)

1.2 Design philosophy

Three principles guided every decision:

- **KISS first, robustness second.** We built a deliberately simple, correct V1 (no fault tolerance), measured it, and only then layered the fault-tolerance protocol (V2) *on top without changing the happy path*.
- **No external dependencies, no privileges.** The engine is pure Python standard library (NumPy is an optional accelerator). Everything runs as an unprivileged user: no Hadoop, no HDFS, no Docker, no root, no ZooKeeper.
- **Measure, never assume.** Every phase is timed and emitted as machine-parsable log lines; the bottleneck and the Amdahl serial fraction are *measured*, and optimisations are justified by before/after numbers.

1.3 Why Python

We chose Python deliberately, not by default. It is one of the most widely used languages in the world today — and the default choice for data processing and large-scale text/ML pipelines, exactly the domain a Common Crawl word-count belongs to. Building the engine in Python let us ask a pointed question: *how far can we push an “interpreted, GIL-bound, slow” language on a genuinely parallel workload before it becomes the bottleneck?* That framing turned a potential weakness into one of the most interesting parts of the project. The GIL forced our per-worker parallelism to be process-level (which maps cleanly onto the distributed model), and the interpreter overhead made

the MAP compute the measured bottleneck — which is precisely what drove the optimisation work in §8 and let us quantify the remaining gap with a C++ prototype ($\sim 1.7\times$ compute). In short: Python maximised readability and development speed, kept the standard-library-only constraint trivial to honour, and gave us a concrete, honest baseline to optimise against and reason about.

1.4 Repository map

Path	Contents
src/mapreduce/	master.py, worker.py, validate.py, download_commoncrawl.py
src/server/, src/client/	Day-1 CPU-load server and parallel client
src/deploy/	deploy_commoncrawl.sh, kill_commoncrawl.sh, fault_tolerance_demo.sh, get_machines.sh, py
src/kafka/	deploy_kafka.sh, run_wordcount.sh, commoncrawl_source.sh, clean_kafka.sh
src/benchmarks/	amdahl_bench.py, plot_amdahl.py, ssh_parallelism_sweep.py, find_scratch.sh, local_cluster.sh
tests/run_all.py	One-command single-machine harness (correctness + FT + Amdahl)
doc/	Guidelines, MapReduce paper, space-time SVGs, optimisation log
report/	This report and the generated Amdahl figures

1.5 Contributions and collaboration

The project was built by a team of four; the work split roughly along the chronological phases of the project.

Member	Main responsibilities
Rémi Deloye	First part of the project: deployment scripts, and the initial implementation and testing of the MapReduce engine (MAP/REDUCE cycle, barrier, shuffle).
Sébastien Rivière	First part of the project (with Rémi): deployment scripts, and implementing and testing the core MapReduce engine.
Camil Daou	Conception of the sequence diagrams; the <code>bench.py</code> measurement library and the Amdahl study; bottleneck testing and the worker optimisations (combine + bytes-regex + vectorised CRC32, parallel SSH shuffle, on-demand ingestion).
Aymen Kabil	Deployment, the fault-tolerance layer (heartbeat/lease, re-execution, straggler backups, atomic commit), the Kafka Streams comparison, and integration of the remaining pieces of the system.

We collaborated through **GitHub**, using separate feature branches per work-stream. We deliberately tried to work in a *sequential* manner — merging one branch before starting the next — to keep the shared `master.py/worker.py` surface conflict-free. This held for most of the project; near the end, with several optimisation, fault-tolerance and benchmarking branches in flight at once, some merge conflicts became unavoidable and were resolved by hand.

2 Warm-up workflow: the CPU-load cluster (Day 1)

Before any MapReduce, Day 1 established the deployment muscle: getting code onto 100 machines and talking to them in parallel.

Machine selection. `get_machines.sh` queries the school’s `tp.telecom-paris.fr/ajax.php` JSON endpoint (the same API the room-status web page uses), filters for reachable hosts, and writes canonical names (e.g. `tp-1a201-05.enst.fr`) to `runtime/machines.txt`.

Server. `src/server/server.py` is a minimal TCP server. On each connection it reads `/proc/loadavg` and replies with the 1-, 5- and 15-minute load averages, then closes. It binds a *dual-stack* IPv6 socket (`IPV6_V6ONLY=0`) with `SO_REUSEADDR` for an instant restart, and shuts down cleanly on `SIGTERM` / `SIGINT` to free the port.

Deployment. Because `$HOME` is on NFS, a *single* SCP copies the server into the shared home; then one SSH per machine launches it. The parallel client (`src/client/client.py`) fans out with a `ThreadPoolExecutor`, collects every node's load, and prints per-node and cluster-wide averages.

Clean-up. `kill_cpuload.sh` kills the listener on the chosen port and removes deployed files on every machine, essential for redeploying a new version cleanly.

This workflow is the template the MapReduce deployment later reuses: *NFS for distribution, SSH for control, parallel fan-out, idempotent clean-up.*

3 System architecture

The engine is a classic **Main / Workers** (master/worker) MapReduce, modelled directly on Figure 1 of Dean & Ghemawat. The Main owns scheduling and never touches the data; Workers do all I/O and computation.

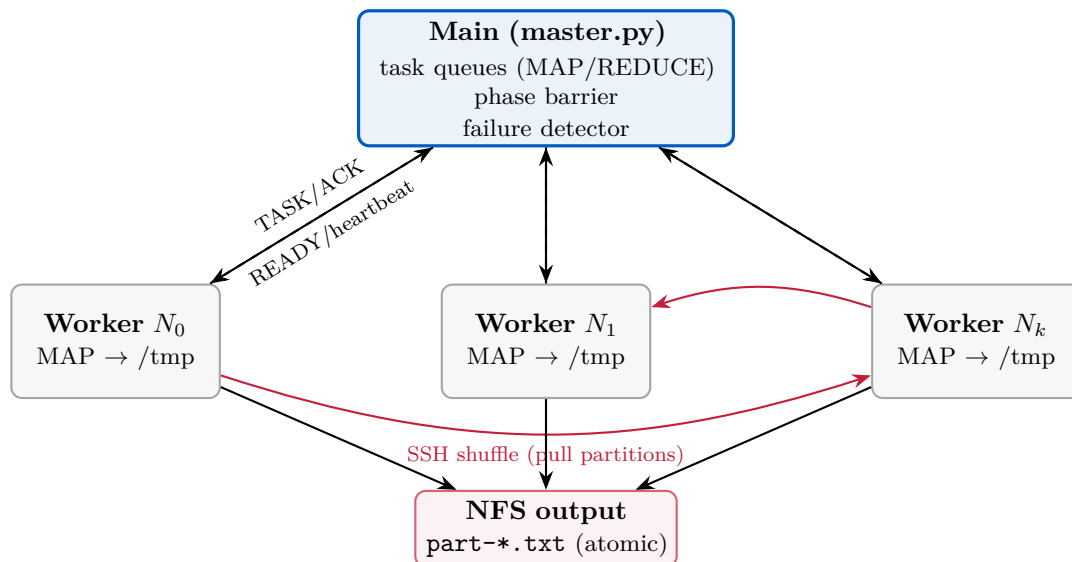


Figure 1: Main/Workers architecture. The Main routes *metadata only* (TCP, JSON lines). MAP intermediates stay on each worker's *local* disk; reducers pull partitions worker-to-worker over SSH; final output is committed atomically to shared NFS.

3.1 Components

- **Main** (`master.py`) holds the MAP and REDUCE queues, drives the MAP→REDUCE barrier, tracks which worker holds which partition, detects failures, and reassigns work. It binds a dual-stack IPv6 socket *before* loading tasks so early workers can connect and wait.
- **Workers** (`worker.py`) register a unique identity, request work in a pull loop, execute MAP or REDUCE, and report completion. A background thread heartbeats the Main. Workers are identified by a `worker_id` (not just host), which lets several run on one machine, the basis of the single-machine test mode.

3.2 Storage model: NFS vs local disk

The school's \$HOME is on NFS; the paper insists MAP intermediates go to *local* disk. We honour both:

Data	Location	Rationale
Input splits (Common Crawl WET)	NFS, spill dir, <i>or</i> in-memory stream	shared input, or bypass NFS entirely
MAP intermediate partitions	local /tmp (or --spill-dir)	writing to NFS would hammer the shared server
REDUCE final output part-*.txt	NFS (shared)	collected in one place, written <i>atomically</i>
Logs	local /tmp/.../logs/<ts>	per-run, off-NFS

Because /tmp can itself be a small RAM-backed tmpfs that saturates under hundreds of concurrent MAP downloads, `src/benchmarks/find_scratch.sh` inspects `df/mount`, excludes NFS and pseudo-f filesystems, and recommends the largest *local writable* partition; the worker's `--spill-dir` then redirects staging there.

4 Network protocol design

4.1 Transport and message set

Transport is **TCP**, one persistent connection per worker. Messages are **newline-delimited JSON** (`{...}\n`); both sides keep a receive buffer to reassemble partial reads. The full message set:

Direction	Message	Purpose
W → M	<code>{"status":"REGISTER","worker_id":...,"host":...,"map_dir":...}</code>	announce identity + local map dir
W → M	<code>{"status":"READY_FOR_TASK"}</code>	request work
W → M	<code>{"status":"TASK_FINISHED"}</code>	report completion
W → M	<code>{"status":"HEARTBEAT"}</code>	liveness ping (every 2s)
M → W	<code>{"type":"MAP","split_id":i,"n_reducers":r,"job":J,"url":...}</code>	assign a MAP
M → W	<code>{"type":"REDUCE","reducer_id":j,"map_sources":[...],"job":J}</code>	assign a REDUCE
M → W	<code>{"type":"WAIT"}</code>	nothing to do yet
M → W	<code>{"status":"ACK"}</code>	acknowledge a finish

A key design choice: **heartbeats are one-directional** (worker → Main). This keeps the Main→worker stream carrying only tasks and ACKs, so the existing framing is untouched by fault tolerance.

4.2 V1: the KISS protocol (no fault tolerance)

The first version implements the bare cycle MAP → barrier → SHUFFLE+REDUCE. Each map worker writes one `partition_j.txt` per reducer to local disk. At reduce time, reducer *j* fetches its partition file from *every* map worker via `ssh cat` and sums.

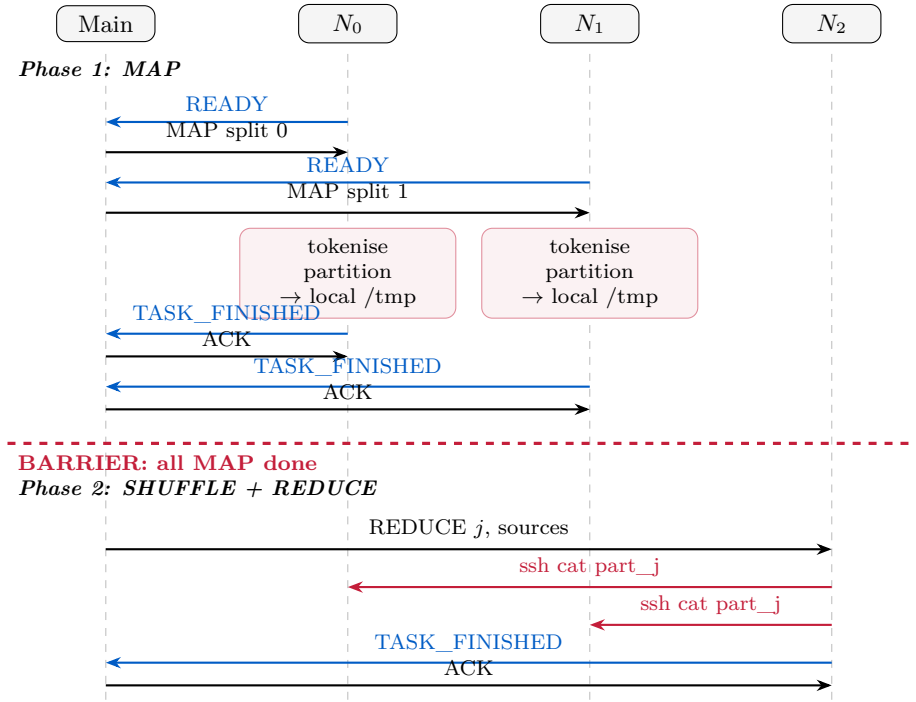


Figure 2: V1 (KISS) space-time diagram. The reducer for key k is $\text{crc32}(k) \bmod R$. The Main only exchanges metadata; the actual partition bytes flow worker-to-worker during SHUFFLE. (Full source rendered on sequencediagram.org as `doc/map_reduce/MapReduce.svg`.)

4.3 Reducer key distribution

Every analysis emits `(key, count)` pairs. The reducer responsible for a key is chosen by hashing, exactly as in the paper:

$$\text{reducer}(k) = \text{crc32}(k) \bmod R.$$

Because the same hash is used by every mapper, all occurrences of a key land in the same reducer, so reduce outputs have disjoint key sets and never need re-summing across reducers.

4.4 Bootstrap

A subtle ordering problem: workers may connect before the Main has finished fetching `wet.paths.gz` and building its queues. The Main solves this by *binding and listening before loading tasks*; any worker that connects early is parked on `WAIT` and retries until the queues are populated. This gives a clean, race-free start-up.

5 Common Crawl ingestion

Common Crawl publishes monthly crawls as WARC/WAT/WET files on `data.commoncrawl.org`; we use the plain-text **WET** extracts. The file list for a crawl is `crawl-data/<CRAWL_ID>/wet.paths.gz` (default crawl `CC-MAIN-2024-10`). It matters because Common Crawl is one of the primary data sources used to train modern LLMs.

Three ingestion paths are implemented, from most to least NFS-dependent:

1. **Pre-download to NFS** (`download_commoncrawl.py --missing-only`), idempotent; turns each WET `.gz` into a `commoncrawl-XXXX.txt` split.
2. **On-demand download to the spill dir**, if the NFS split is absent and the MAP task carries a `url`, the worker downloads it to local disk, processes it, then deletes it.

3. **Direct read from Amazon into memory** (`--direct-read`), the worker streams and gunzips the `.wet.gz` straight from S3/HTTPS, strips WARC/HTTP headers on the fly, and processes the bytes *without ever touching NFS or staging to disk*. This is the Day-4 mandate to “retire the internal NFS” that saturates under massive concurrent reads.

6 The four analyses (pluggable jobs)

All analyses share *one* engine. The shuffle and reduce (sum by key, sort descending) are identical; only the MAP keying differs, selected by `-j/--job` and implemented in a single function `_map_emit`. Every job emits bytes keys so the downstream path is uniform.

<code>-j</code>	Analysis	MAP key	Question answered
<code>wordcount</code>	word frequency	<code>word</code> $\rightarrow$ 1	which words are most frequent?
<code>lang</code>	language ranking	<code>stop-word hit</code> $\rightarrow$ <code>language</code>	which languages dominate the crawl?
<code>wordlen</code>	length distribution	<code>len(word)</code> $\rightarrow$ 1	what is the word-length histogram?
<code>bigram</code>	phrase popularity	$w_1 w_2 \rightarrow 1$	which two-word phrases are most common?

Methods. `lang` matches each token against tiny, high-frequency stop-word sets for en/fr/de/es/it; a token that is a stop-word in exactly one language is a strong signal (tokens shared by several languages are dropped to avoid bias). `wordlen` keys on `len(word)`; `bigram` keys on consecutive word pairs $w_1 w_2$.

What a bigram is, and “function-word pairs dominate”. A *bigram* is simply a pair of adjacent words in the text: sliding a two-word window over “the cat sat on” yields the bigrams *the cat*, *cat sat*, *sat on*. Counting them tells us which two-word phrases co-occur most often. The result is always led by *function words*, the grammatical “glue” of a language (articles, prepositions, conjunctions, pronouns: *the*, *of*, *in*, *to*, *and*), as opposed to *content words* (nouns, verbs) that carry topic meaning. Because a handful of function words make up a huge share of all tokens, the pairs they form (*of the*, *in the*, *to the*) vastly outnumber any specific content phrase. So “function-word pairs dominate” means the top of the bigram ranking is grammatical glue, not topical phrases. It is a useful signal for spotting templated/boilerplate text and a first step toward *n*-gram language modelling.

6.1 Observed results on a real Common Crawl split

The tables below are **actual output** produced by the engine’s own `_map_emit` logic on a genuine Common Crawl split, the first WET file of crawl CC-MAIN-2024-10 (split `...-00000.warc.wet.gz`), ≈ 62.9 MB of decompressed text, **6,622,954 tokens**. Each analysis is also checked for internal consistency against the single-machine reference (§12).

lang: language ranking. Of the 611,007 single-language stop-word hits, **English dominates with $\approx 79\%$** , followed by the major European languages in the expected order:

rank	language	stop-word hits	share
1	en (English)	483,085	79.06%
2	fr (French)	65,300	10.69%
3	de (German)	33,684	5.51%
4	es (Spanish)	19,048	3.12%
5	it (Italian)	9,890	1.62%

Table 1: `lang` observed result: English leads by a wide margin: exactly Common Crawl’s published language distribution, confirming the tokeniser sees real multilingual web text.

wordlen: word-length distribution. The histogram is *unimodal, peaking at 2–4 characters* and decaying with a long tail (lengths run up to 161), the expected natural-language shape:

length	2	4	3	5	1	6	7	8
share	16.0%	15.5%	13.1%	12.3%	11.0%	8.6%	7.1%	5.9%

Table 2: `wordlen` observed result: short tokens dominate; lengths 2–5 alone account for $\approx 57\%$ of all tokens. The very long “words” in the 161-bucket tail are non-text artefacts (URLs, hashes) in the WET extraction.

wordcount: word frequency. 435,471 distinct words; the head is the usual English function words, mixed with multilingual tokens (`de`) and some residual WET record-metadata tokens (`warc`, single digits):

word	count	word	count	word	count	word	count
the	79,745	a	76,287	to	58,487	and	49,884
of	48,187	in	42,100	de	47,992	warc	48,421

Table 3: `wordcount` observed result: *the/a/to/and/of/in* top the list as expected. `de` reflects French/Spanish content; `warc` and bare digits are residual WET metadata (see the note below).

bigram: phrase popularity. 2,467,306 distinct pairs. Genuine English collocations (*of the*, *refers to*) appear at the top alongside WET record-metadata pairs:

bigram	count	bigram	count
of the	9,984	refers to	6,948
urn uuid	13,849	warc record	6,925

Table 4: `bigram` observed result: *of the* / *refers to* are real high-frequency English phrases; *urn uuid* / *warc record* / *2024 02* are WET-header metadata pairs.

A note on metadata noise. The `warc`, `urn uuid` and `warc record` entries are not English, they are tokens from the WET *record-metadata* headers (`WARC-Type`, `WARC-Record-ID`: `<urn:uuid:...>`) that our deliberately lightweight header strip (prefixes `WARC-`, `Content-`, `Metadata-`) leaves behind. This is exactly the “simple parsing $\Rightarrow$ minor metadata pollution” trade-off flagged in the pain points (§15): `lang` and `wordlen` are unaffected (they key on languages / lengths), while `wordcount` and `bigram` surface it. A stricter `WARC-` prefix filter removes it at the cost of a little extra parsing, a clean illustration of the cost/precision trade-off in real-world data cleaning.

7 The `bench.py` measurement library

Every optimisation in §8 is justified by a before/after number, and every one of those numbers had to be *trustworthy*. That guarantee comes from `src/benchmarks/bench.py`, a small (~380-line, zero-dependency) statistical timing library we wrote before touching a single hot loop. It is the instrument the rest of the performance story is measured with, so it earns its own section.

7.1 Why we built it (and why not a bare chronometer)

The naïve way to time code is a single `time.time()` delta. We rejected it for three concrete reasons:

- **`time.time()` is not monotonic.** It follows the wall clock, so an NTP correction or a DST jump mid-measurement silently corrupts the delta. `bench.py` uses `time.perf_counter_ns()` — a monotonic, nanosecond-resolution clock that never runs backwards.
- **One number is not a measurement.** A phase that runs many times (one TCP query per node, one MAP per split) has a *distribution*, not a value. A single sample cannot tell a stable 30 ms from a 30 ms mean hiding a 200 ms tail. We needed variance, percentiles, and a stability signal.
- **“Optimise the bottleneck” requires knowing the bottleneck.** Guessing which phase dominates is how projects waste effort tuning code that is 2% of the wall. We wanted the tool itself to *attribute* time to phases and categories and name the culprit.

In short, we wanted the guarantees a tool like *Google Benchmark* gives, but embedded in the project and aware of our own phase taxonomy.

7.2 How it works

The library is built around a `Registry` that stores every timed span as a *sample* keyed by a label, and exposes three ways to feed it plus a reporting/export back end.

- `phase(label, category)` — a context manager that times an arbitrary block and records one sample.
- `@timed(...)` — a decorator that records one sample per call of a function.
- `benchmark(label, fn, min_time, warmup)` — the Google-Benchmark micro-benchmark loop: it runs `warmup` discarded iterations (to page in code and warm caches), then calls `fn` repeatedly, recording *every* iteration as a sample, until a wall budget of `min_time` seconds has accumulated. This auto-scales the iteration count so a 1 µs kernel and a 1 s kernel are both measured to the same statistical confidence.

Each sample carries a **category** — `io`, `network`, `compute`, `sync`, `deploy`, `cleanup` — which is exactly the Day-4 breakdown the brief asks for. A representative use, comparing the MAP tokeniser candidates that became Optimisation 1:

```
import bench

raw = open("commoncrawl-0000.txt", "rb").read()

# auto-scaled micro-benchmarks: warm up, then loop until 0.5s of samples
bench.benchmark("map:baseline",      lambda: baseline_split_isalnum(raw))
bench.benchmark("map:bytes-regex",   lambda: _WORD_RE.findall(raw))
bench.benchmark("map:combine+crc32", lambda: map_emit(raw, n_reducers))

bench.report(title="MAP tokeniser candidates") # ranks phases, names bottleneck
bench.save_json("map_bench.json")             # raw samples for plotting
```

From the collected samples, `stats()` computes a full `Stats` record per label:

Metric	Why it is reported
count, total	how many samples, and their summed time (drives the % ranking)
mean, median	central tendency; median resists outliers the mean does not
p90, p95	tail latency — what a “slow” occurrence actually costs
stdev, min, max	spread of the distribution
$CV = \sigma/\mu$	dimensionless <i>stability</i> : the library flags any phase with ≥ 5 samples and $CV > 0.5$ as high variance

Table 5: The per-phase statistics `bench.py` reports. Percentiles use linear interpolation over the sorted samples.

`report()` then sorts phases by total time, prints a per-category roll-up, and explicitly names the **bottleneck phase** and **bottleneck category** — so the tool answers “what should I optimise?” directly rather than leaving it to eyeballing. `save_json()` dumps the raw per-sample series (clock, unit, samples, and the derived stats), which is the format the plotting scripts (§9) consume.

7.3 How it was used, and how it helped

`bench.py` was the decision procedure behind the optimisation work, not an afterthought:

- **It located the bottleneck first.** Before optimising, the category roll-up showed `compute` at **>85%** of worker time, proving the MAP tokeniser — not I/O or network — was the thing to attack. That single result is what justified spending effort on the bytes-regex + combine + vectorised CRC32 of Optimisation 1 (§8) instead of, say, the shuffle.
- **It compared candidates fairly.** The auto-scaling `benchmark()` loop let us race `split()+isalnum()` against the bytes-regex and the combine path on the *same* input to the same time budget, so the $1.52\times$ compute speed-up is a measured ratio of stable means, not a lucky single run.
- **It told us when a number was noise.** The CV / high-variance flag kept us honest: network- and shuffle-category phases that showed $CV > 0.5$ were reported as unstable, which is precisely why the shuffle tuning was validated with a dedicated sweep (`ssh_parallelism_sweep.py`) rather than a single timing.
- **It produced the evidence.** The JSON export is the raw material behind the tables and figures in §8 and the Amdahl study (§10); the same sample-based, per-phase, per-category methodology is mirrored by the distributed `WORKER_TIMING` instrumentation (§9) that measures the real cluster.

The payoff is a project where “we optimised X and it got faster” is never a claim of faith: every such statement traces back to a distribution of samples collected by this library.

8 Optimisations

We started from a deliberately naïve worker, measured it, and optimised the *proven* bottleneck — never a guessed one. Every number below comes from the raw per-phase benchmark logs (`amdahl_results_{python,opt1,opt2,cpp}.json`) captured on the *same* 32-split, 8-reducer workload, so all four worker builds are directly comparable row-by-row. The methodology (warm-up, fixed-budget timing, the speed-up and Karp–Flatt derivations) is detailed in §10; here we report the outcomes.

8.1 Baseline (naïve worker)

`line.split()` + `isalnum()` tokenisation; `crc32` called once *per word occurrence*; UTF-8 text I/O; one word\t1 line per occurrence; *sequential* SSH shuffle. Profiling showed `t_compute` is >85% of worker time, the MAP loop is the bottleneck, not I/O or network.

8.2 Optimisation 1: combine + bytes-regex + vectorised CRC32

- **Bytes-regex tokeniser** `re.compile(rb'[A-Za-z0-9]+')`: a single pass over raw bytes, no UTF-8 decode, no per-line loop.
- **Combine before partition**: aggregate *all* occurrences into one dict first, so `crc32` is called once per *unique* key, not per occurrence.
- **Vectorised CRC32**: `np.frompyfunc(zlib.crc32, 1, 1)` over the array of unique keys (with a pure-Python fallback when NumPy is absent).
- **Batched writelines** instead of one write per word.

The three ideas are visible in `src/mapreduce/worker.py` (`_map_emit` and `_execute_map`):

```
# 1. tokenise raw BYTES in one pass: no UTF-8 decode, no per-line loop
_WORD_RE = re.compile(rb'[A-Za-z0-9]+')
tokens    = _WORD_RE.findall(raw_data)

# 2. COMBINE: fold every occurrence into a dict -> one entry per UNIQUE key
counts = {}
for tok in tokens:
    k = tok.lower()
    counts[k] = counts.get(k, 0) + 1

# 3. vectorised CRC32 over the UNIQUE keys only (pure-Python fallback if no NumPy)
_crc32_vec = np.frompyfunc(zlib.crc32, 1, 1)
rids = _crc32_vec(keys).astype(np.int64) % n_reducers
```

N	base t_{total}	opt1 t_{total}	base compute	opt1 compute	speed-up (total)
1	275.4	174.4	234.9	156.2	1.58×
2	148.2	99.3	233.8	154.6	1.49×
4	92.8	68.7	235.4	154.4	1.35×
8	53.5	40.6	235.7	154.8	1.32×
16	55.4	45.2	235.6	154.6	1.23×
32	61.2	47.7	235.9	154.8	1.28×

Table 6: Baseline vs Optimisation 1 (seconds, 32 splits). Compute is a consistent **1.52×** faster. As a free bonus, the combine phase shrinks each partition to one line per *unique* word, cutting `t_shuffle` **4–5×**.

8.3 Optimisation 2: parallel SSH shuffle with connection multiplexing

The reducer fetches all partitions concurrently with a `ThreadPoolExecutor`, turning shuffle wall-time from $O(N \cdot \text{latency})$ into $\approx O(\text{latency})$. To avoid tripping the cluster’s `fail2ban`/IDS with a burst of SSH connections, we enable `ControlMaster=auto` multiplexing: the first SSH to a host opens one real TCP connection and all subsequent fetches (even from other reducers on the same machine) reuse it. A sweep (`ssh_parallelism_sweep.py`) showed shuffle wall-time *floors* at `max_ssh` ≈ 6 –8 for $N \leq 32$, so the default is `min(n_workers, 8)`.

The shuffle loop in `worker.py` (`_execute_reduce`):

```
# one real TCP connection per host, reused by every later fetch
ssh_opts = ("-o ControlMaster=auto "
            "-o ControlPath=/tmp/ssh-ctrl-%h-%p -o ControlPersist=120s")

# fetch all partitions concurrently: wall-time ~ slowest single fetch
with ThreadPoolExecutor(max_workers=min(len(sources), 8)) as ex:
    futures = {ex.submit(fetch_partition, s): s for s in sources} # ssh host 'cat
    partition_j'
    for fut in as_completed(futures):
        all_raw_lines.extend(fut.result()[1])
```

The direct effect is visible in the `t_shuffle` column: the sequential reducer paid a shuffle cost that grew with N (more mappers to pull from, one at a time), whereas the parallel reducer’s shuffle wall-time stays flat and small because it equals the *slowest single* fetch, not their sum.

N	opt1 t_{shuffle}	opt2 t_{shuffle}	opt1 t_{total}	opt2 t_{total}
1	4.8	1.2	174.4	127.7
2	9.4	1.4	99.3	64.6
4	16.8	2.1	68.7	39.9
8	27.6	3.9	40.6	35.1
16	28.2	2.8	45.2	39.4
32	26.9	4.8	47.7	36.4

Table 7: Optimisation 1 vs Optimisation 2 (seconds, 32 splits). The `t_shuffle` column isolates the parallel-SSH change: a 5–8× reduction that no longer scales with the number of mappers. `t_total` also improves because Opt2 is measured on the fully-tuned Python worker.

8.4 What the optimisations moved — and what they did not

A subtle but important point for the Amdahl discussion (§10): the optimisations **clearly lower the absolute wall-clock time**, but they **do not improve — and in fact reduce — the parallel speed-up ratio**. The two are not contradictory; they are the expected consequence of Amdahl’s law.

worker build	$t_{\text{total}}(1)$	best t_{total}	peak speed-up	Karp–Flatt e ($N=8$)
Baseline (naïve)	275.4	53.5 ($N=8$)	5.15×	0.079
Opt1 (compute)	174.4	40.6 ($N=8$)	4.30×	0.123
Opt2 (+parallel shuffle)	127.7	35.1 ($N=8$)	3.64×	0.172
C++ worker (prototype)	132.5	35.5 ($N=16$)	3.73×	0.180

Table 8: Scalability summary across worker builds (32 splits). The absolute wall drops **53.5→40.6→35.1** s, yet peak speed-up *falls* (5.15→3.64×) and the Karp–Flatt serial fraction *rises* (0.079→0.172).

Why the wall moves but the speed-up shrinks. The Karp–Flatt metric $e = \left(\frac{1}{S_p} - \frac{1}{p}\right) / \left(1 - \frac{1}{p}\right)$ estimates the *experimentally serial fraction*. Each optimisation attacks the *parallel* part (per-split MAP compute), so it shrinks the numerator of the parallel work while the fixed serial/overhead cost (job setup, the final merge, and especially shuffle/NFS traffic) stays roughly constant. By Amdahl, making the parallel part cheaper *raises* the serial *fraction* of what remains — hence a

smaller speed-up even though every configuration finishes sooner in absolute terms. This is the correct, healthy outcome: we were paid in latency, not in a better slope.

The true ceiling is NFS-read contention, not a serial code path. All builds reach their floor at $N \approx 8$ and then *regress* (e.g. Opt1: 40.6s at $N=8$ but 47.7s at $N=32$). The cause is visible in the phase logs: t_{io_read} grows with N (Opt1 rises from 3.4s at $N=1$ to **69s** at $N=32$) because every worker reads its split from the shared NFS home at once. This is an *overhead* wall, not a fixed serial fraction, and it is precisely what the on-demand Common Crawl ingestion (§5) removes by having each worker pull its own split directly from Amazon into local `/tmp`.

8.5 Optimisation 3: on-demand ingestion — and its superlinear scaling

The mechanism is described in §5 (path 2): when the master ships a `url` with each MAP task, the worker downloads its split to local `/tmp`, processes it, and deletes it — the shared NFS home is never touched on the read path. We benchmarked it on the real 32-split CC-MAIN-2024-10 crawl to confirm it removes the contention wall above, and it produced the most interesting curve in the whole study: a **superlinear** speed-up ($S(N) > N$).

N	t_{total}	speed-up	ideal	$t_{dl,max}$	t_{io_read}
1	1646.6	1.00×	1×	172.2	16.1
2	603.7	2.73×	2×	9.0	2.2
4	307.8	5.35×	4×	8.6	1.5
8	199.1	8.27×	8×	11.6	1.6

Table 9: On-demand ingestion (seconds, 32 splits, CC-MAIN-2024-10). Speed-up *exceeds* ideal-linear at every point. $t_{dl,max}$ is the longest single split download and t_{io_read} the local-disk read time.

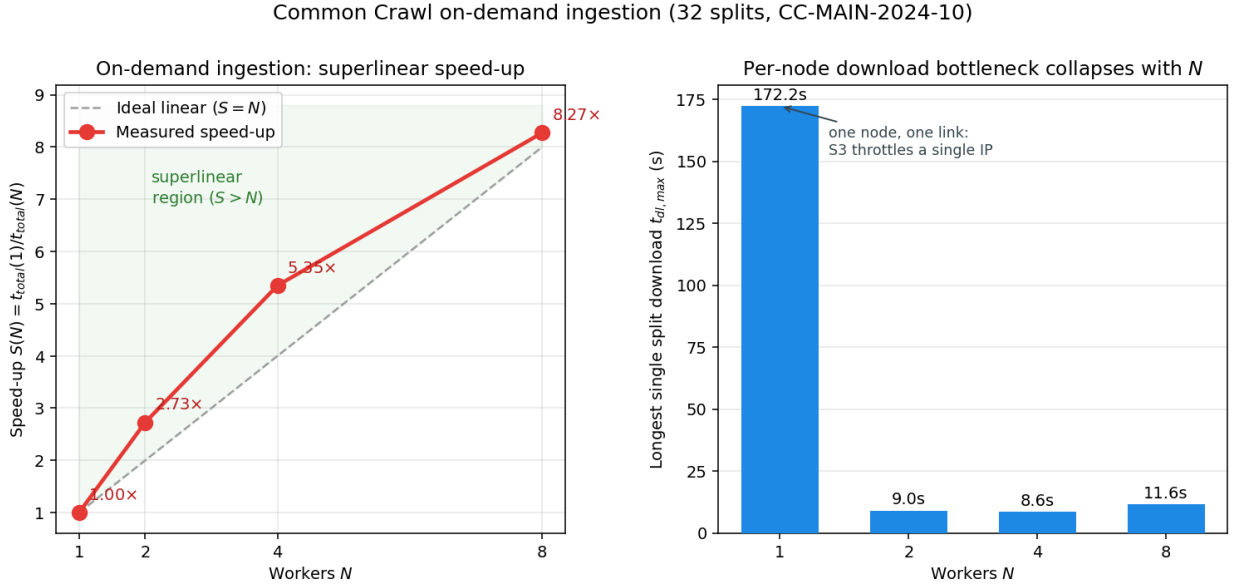


Figure 3: On-demand ingestion. Left: the measured speed-up sits *above* the ideal-linear diagonal (the shaded superlinear region), reaching 8.27× at $N=8$. Right: the mechanism — the longest single split download ($t_{dl,max}$) collapses from 172s at $N=1$ (one node, one throttled S3 link) to ≈ 9 s the instant a second node shares the load, so adding workers relieves a per-node bottleneck at the same time as it divides the work.

What it fixes. The read path no longer goes through NFS, so `t_io_read` stays flat at ≈ 1.5 s for $N \geq 2$ instead of climbing to 69 s (§8.4); the ~ 5 GB NFS quota and the separate pre-staging step both disappear, and each node only ever fetches the handful of splits it will actually process.

Why the curve goes superlinear (the advantage). Amdahl assumes the work and the machine are fixed, so $S(N) \leq N$. Here the *per-node* resource budget changes with N . At $N=1$ a single node must pull all 32 splits over *one* network link and one disk: the longest single download alone is **172 s** (one IP hammering S3 gets throttled), and the serialised downloads dominate the 1647 s baseline. Add nodes and the aggregate download bandwidth and the number of independent local disks both grow with N , so `t_dl,max` collapses from 172 s to ≈ 9 s the moment a second machine joins. The parallelisable work is therefore divided by N *while* a per-node bottleneck is simultaneously relieved — the two effects compound and push $S(N)$ above the diagonal.

The setbacks. The same physics that makes the curve pretty makes it dishonest as an efficiency metric, and expensive at small N :

- **A pathological serial baseline.** The superlinear number is inflated by a crippled $N=1$ (1647 s here vs ≈ 61 s for the pre-staged path in §8.4). It measures *escape from a resource bottleneck*, not genuine compute parallelism, so it should not be read as “better than perfect scaling”.
- **Amdahl no longer bounds it.** Because per-node bandwidth and cache change with N , the fixed-work model does not apply; the curve cannot be extrapolated with a single serial fraction f the way the pre-staged sweeps can (§10).
- **External dependency and non-reproducibility.** Wall-time is now hostage to S3 bandwidth and throttling; `t_download` varies run-to-run and the experiment is no longer self-contained.
- **Re-download on failure.** A re-executed MAP (§11) must re-fetch its split from the internet, since nothing is cached on NFS.

The takeaway: on-demand ingestion is the right choice at scale (it removes the NFS wall and lets download I/O parallelise across nodes), but its superlinear speed-up is a symptom of a resource-starved baseline, not a free lunch.

Future ideas (logged, not all benchmarked): gzip partitions before transfer, *push* partitions after MAP instead of pulling, replace SSH with raw TCP, and finishing the C++ worker (the `cpp-worker` prototype already gives $\sim 1.7\times$ compute over the tuned Python worker, confirming the remaining cost is the interpreter, not the algorithm).

9 Performance instrumentation

Every phase of the lifecycle is timed and emitted as a structured, grep-able log line, satisfying the Day-4 requirement to time deploy, clean, I/O, sync/wait, network, and compute separately.

- **Main** emits TIMING: `{"t_total", "t_map", "t_reduce"}`.
- **Workers** emit one WORKER_TIMING line per task: `t_download`, `t_clean`, `t_io_read`, `t_compute` for MAP and `t_shuffle`, `t_compute`, `t_io_write` for REDUCE.

Two harnesses parse these lines into `runtime/amdahl_results.json`, which `plot_amdahl.py` renders:

- `tests/run_all.py`, a **single-machine** harness (several worker processes on `localhost`, local-disk shuffle) that reproduces the *entire* pipeline (correctness, fault tolerance, and the Amdahl sweep) with one command and no cluster.

- `src/benchmarks/amdahl_bench.py`, the **multi-node cluster** sweep ($N = 1, 2, 4, 8, 16, \dots$ real machines over SSH), same methodology, larger N .

The **measured bottleneck** is unambiguous: on the baseline worker `t_compute` is over 85% of worker time; on the cluster the per-worker MAP compute dominates and the REDUCE/shuffle is comparatively small (Fig. 4, right panel).

10 Amdahl’s law: empirical validation

10.1 Method and the correctness rule

The reference point is $N = 1 \Rightarrow S = 1$, and speedup is $S(N) = t_{\text{total}}(1)/t_{\text{total}}(N)$. The *same dataset is reused at every point*: changing the data between points would invalidate the comparison, as the brief stresses. We only interpolate under the linear-scaling hypothesis on a *single* machine; with ≥ 2 nodes we *measure*. Amdahl’s law states

$$S(N) = \frac{1}{f + \frac{1-f}{N}}, \quad \lim_{N \rightarrow \infty} S(N) = \frac{1}{f},$$

where f is the incompressible serial fraction.

10.2 Cluster results (real machines over SSH)

The headline result is the multi-node sweep on 16 real Common Crawl splits (8 reducers), the same dataset at every N :

N	t_{total} (s)	t_{map} (s)	t_{reduce} (s)	speedup
1	521.66	490.55	31.11	1.00×
2	262.80	245.26	17.54	1.99×
4	137.15	126.89	10.26	3.80×
8	84.60	77.69	6.91	6.17×
16	67.37	60.99	6.38	7.74×

Table 10: Cluster Amdahl sweep (16 splits, 8 reducers, same dataset throughout).

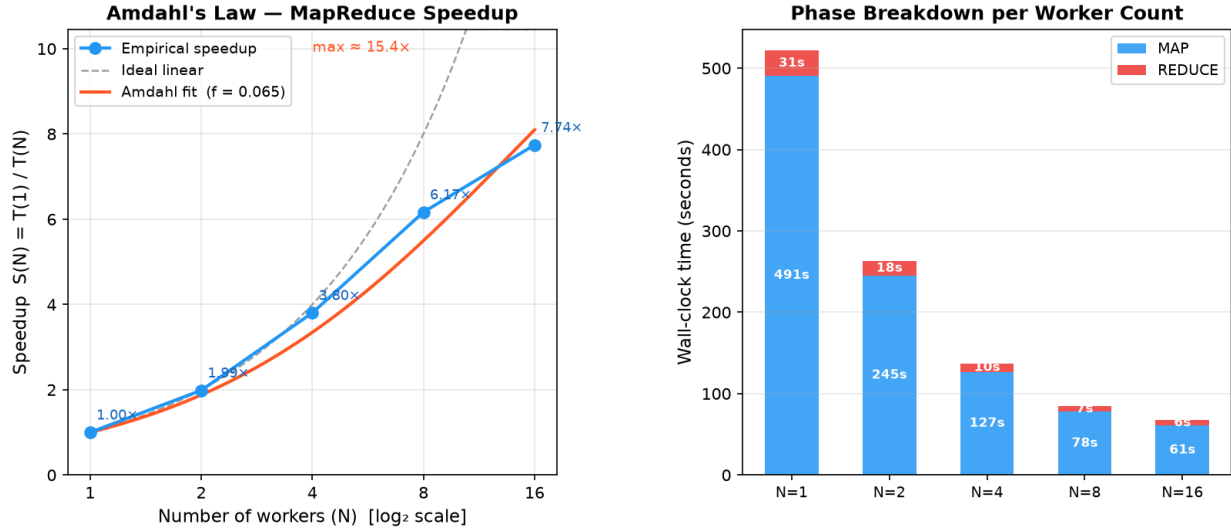


Figure 4: Cluster Amdahl’s law. Left: measured speedup vs ideal-linear, with a fitted serial fraction $f = 0.065$ implying a theoretical ceiling of $\approx 15.4\times$. Right: per-worker-count wall-clock split into MAP (blue) and REDUCE (red), showing MAP dominates and shrinks near-linearly with N .

The measured curve tracks the Amdahl fit closely: $1.99\times$, $3.80\times$, $6.17\times$, $7.74\times$ at $N = 2, 4, 8, 16$. The fitted $f = 0.065$ is small because the workload is embarrassingly parallel in its MAP phase; the residual serial part is the MAP→REDUCE barrier, the portion of shuffle that cannot overlap, master bootstrap, and final output collection. As N approaches the number of splits, the marginal gain shrinks (there is simply less MAP work to hand out), which is exactly why the empirical curve bends below ideal-linear at $N = 16$.

Why the sweep stops at 16 and not at 100. The lab platform has $100+$ machines, and the engine can address all of them: Day 1 deploys the load-average server to the full pool (§2) and the Main can hand tasks to as many workers as we register. The Amdahl study is nonetheless capped at $N = 16$ on purpose, because it is a *strong-scaling* experiment: the rule from the brief is that **the dataset must stay identical at every point**, and our fixed benchmark is cut into **16 splits** (one MAP task each). With 16 MAP tasks, any worker beyond the 16th simply has no MAP work to claim and sits idle, so $N > 16$ *cannot* raise the MAP speedup, it only adds scheduling and barrier overhead and would bend the curve further below ideal. The two numbers therefore describe different things and are consistent: **100** is the *platform capacity* (and what weak scaling, growing the data and the workers together, would exploit), whereas **16** is the largest worker count at which *this fixed dataset* still yields meaningful parallelism. Pushing N to 100 here would just plot a flat line past 16, which is why we report the strong-scaling curve only where it is informative.

10.3 Single-machine sweep (reproducible by one person)

For reproducibility without a cluster, `tests/run_all.py` runs the same methodology with several worker processes on one box (8 splits, 8 reducers, fixed synthetic dataset):

N	t_{total} (s)	speedup
1	1.584	1.00×
2	1.006	1.57×
4	0.826	1.92×

Table 11: Single-machine sweep.

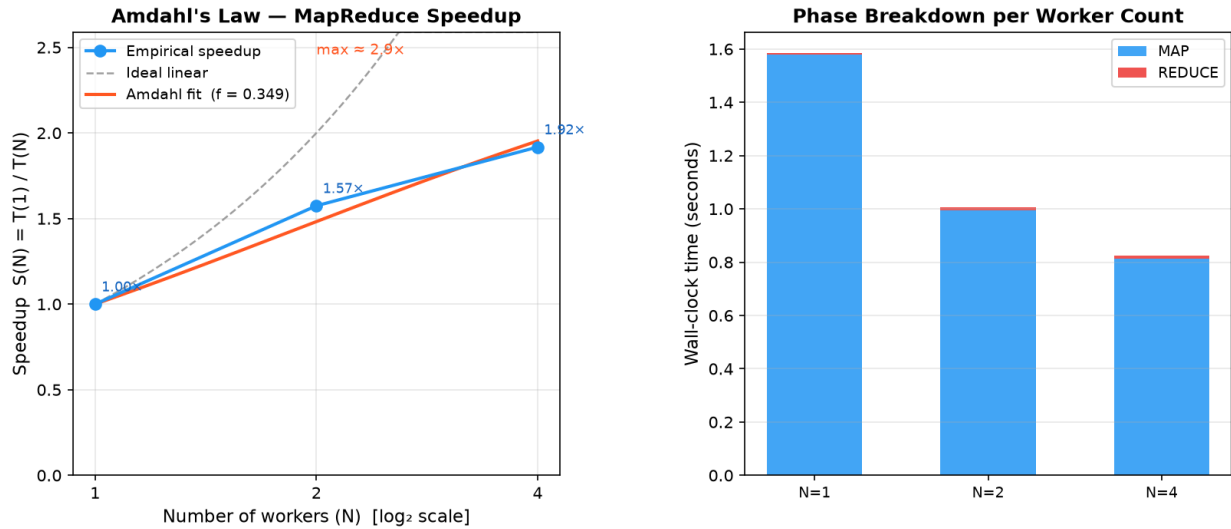


Figure 5: Single-machine Amdahl’s law. The fitted $f = 0.349$ is *larger* than on the cluster: co-located worker processes share one machine’s memory bandwidth and disk, inflating the serial fraction and capping the ceiling near $2.9\times$. This contrast between the two figures is itself the lesson, Amdahl’s f is a property of the *whole system*, not just the algorithm.

10.4 Interpretation

The two figures together tell the full story: on dedicated hardware the engine scales close to linear ($f = 0.065$, $7.74\times$ at $N = 16$); crammed onto a single shared machine it hits a wall near $3\times$ ($f = 0.349$). The per-build optimisations (§8.4) then lower absolute time while *raising* the serial fraction f — and on-demand ingestion (§8.5) even escapes the bound superlinearly — so f is a property of the *whole system* (algorithm, I/O path, and hardware), exactly as Amdahl predicts, demonstrated empirically and reproducibly.

11 Fault tolerance

V1 had a fatal flaw discovered by stress-testing: a *single* dead worker froze the whole job forever (the completion counter never reached the total, so the Main hung and had to be killed by hand). V2 adds five mechanisms from the paper (§3.1–3.6), *without changing the happy path*.

11.1 1. Failure detection (heartbeat + lease)

Workers heartbeat every `HEARTBEAT_INTERVAL = 2 s`. The Main arms a `recv()` lease of `LEASE_TIMEOUT = 60 s` on each connection. Two distinct failures are caught:

- **Process/host crash**, the TCP socket closes, `recv()` returns empty *immediately*, regardless of the lease.
- **Network partition / hang**, no bytes arrive; the lease expires and the worker is declared dead.

The lease (60s) is deliberately well above a single real-split MAP (~ 30 s of GIL-bound pure-Python compute, during which the background heartbeat thread cannot grab the GIL). This avoids wrongly killing a *busy but healthy* worker, while a genuine crash is still detected instantly via socket close.

11.2 2. Task reassignment (re-execution rules)

When a worker dies, `_reclaim()` applies the paper’s rules exactly:

State on dead worker	Action
in-progress MAP or REDUCE	re-queue the task
completed MAP	re-run , its /tmp output died with the machine
completed REDUCE	keep , output was committed atomically to NFS
MAP source dies <i>during</i> REDUCE	revert to MAP phase, re-run lost maps, re-queue <i>all</i> reduces

The last rule is heavy but correct: if a machine that produced a partition dies after the barrier, reducers can no longer fetch from it, so the Main rebuilds the shuffle sources by recomputing the lost maps and redoing the reduces (safe because reduce output is idempotent and atomic).

11.3 3. Atomic output commit (exactly-once)

A reducer writes `part-j.<pid>.tmp` and then `os.replace()`s it into `part-j.txt`. Since `os.replace` is an atomic rename on the same filesystem, a reducer that crashes mid-write *never* leaves a half-written file, and a re-executed reducer cleanly overwrites the previous attempt. This is what makes “keep completed REDUCE” safe.

11.4 4. Straggler backup tasks (§3.6)

When the MAP queue is empty but tasks are still in flight, an idle worker is handed a *duplicate* of the slowest in-flight split (older than `STRAGGLER_THRESHOLD = 90 s`). Whichever copy finishes first wins; the loser’s late `TASK_FINISHED` is recognised as stale (the task is already completed) and discarded.

11.5 5. Master recovery (designed, not implemented)

The space-time design (Fig. 6) includes Main checkpointing to NFS and resuming from the last barrier on restart. We state this plainly in the self-assessment: this is *specified in the protocol but not implemented in code*, worker fault tolerance is fully working; master recovery is future work.

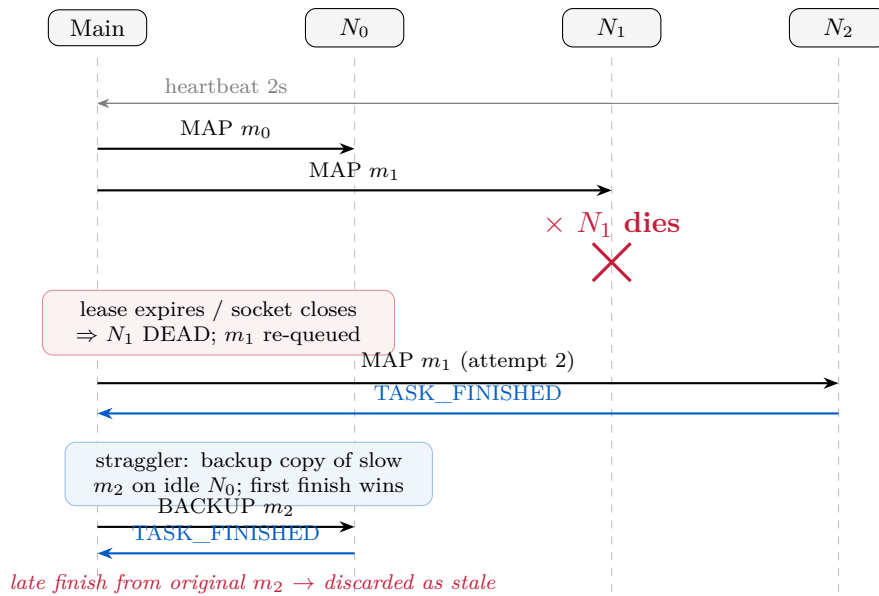


Figure 6: Fault-tolerance protocol (V2), simplified. Heartbeats feed a lease-based detector; a dead worker’s in-progress and completed-MAP tasks are re-executed; stragglers get backup copies. Full source: `doc/fault_tolerance/fault_tolerance.secdiag.txt`.

11.6 Demonstrating it

`src/deploy/fault_tolerance_demo.sh` kills N worker processes mid-job; the Main logs the detection and reassignment, and the job still completes with a correct result. The single-machine `run_all.py` automates this: it kills a worker mid-run and asserts the final output still matches the reference.

12 Validation

`src/mapreduce/validate.py` is the correctness oracle. It *imports the worker's own `_map_emit`* so the reference computation cannot drift from the distributed one, recomputes the chosen analysis on a single machine over the same splits, and compares against the aggregated `part-*.txt`:

1. number of distinct keys (reference vs distributed),
2. sum of all counts,
3. per-key equality (every key has the same count on both sides).

It prints the top- N and exits 0=PASS / 1=FAIL / 2=error. This is the “run a single-machine word count on the same dataset and confirm identical counts” check from the brief, generalised to all four jobs.

13 Kafka Streams comparison

To contrast our *batch* engine with a *streaming* one, we orchestrate the official Apache Kafka (4.x) in **KRaft mode, no Docker, no root, no ZooKeeper**, all state under `/tmp` so NFS is untouched. We did not write a stream processor; we drive Apache's prebuilt `WordCountDemo` with four small Bash scripts.

Script	Role
<code>deploy_kafka.sh</code>	download + start a single-node KRaft broker on <code>/tmp</code>
<code>run_wordcount.sh</code>	create topics, reset app state, run <code>WordCountDemo</code> , feed input, print top- N
<code>commoncrawl_source.sh</code>	stream a Common Crawl split <i>directly</i> into the input topic (no file)
<code>clean_kafka.sh</code>	stop the app + broker and wipe data (<code>-a</code> for a full wipe)

Why an `awk` post-step. The output topic is a *compacted changelog*: each word is re-emitted with a growing count (`the→1, the→2, ...`). We keep the *last* value per key, then sort:

```
awk -F'\t' 'NF==2 {last[$1]=$2} END {for (k in last) print last[k]"\t"k}' \
| sort -rn | head -n "$TOP_N"
```

Direct Common Crawl → Kafka source. `commoncrawl_source.sh` is a single shell pipeline, `curl | gunzip | grep -v headers | head | kafka-console-producer`, that resolves a split from `wet.paths.gz` and streams it into `streams-plaintext-input` with *zero* intermediate files. It is the streaming analogue of our native `--direct-read`: the batch side streams Amazon → RAM, the stream side streams Amazon → Kafka topic.

Result. The top words (`the, to, of, and, a, ...`) are *identical* to our batch engine's output. The difference is purely operational: Kafka keeps the result live and updates it forever; our job emits one final answer and exits.

14 Comparison: our system vs Hadoop vs Kafka Streams

Batch vs stream in one line. Batch (our system, Hadoop) reads bounded input, runs to completion, emits a final result once. Stream (Kafka Streams) reads unbounded input and produces an incremental result that never “finishes”.

Dimension	Our system	Hadoop MapReduce	Kafka Streams
Model	Batch	Batch	Stream (continuous)
Language	Python (+ Bash)	Java	Java/Scala
Storage	NFS + local /tmp, manual remote read	HDFS (locality, replication)	Kafka topics (log)
Scheduling	our Main, 1 task/worker pull	YARN, speculative exec	broker-side assignment
Fault tolerance	heartbeat + re-exec + atomic writes	re-exec + speculative + HDFS replication	changelog + state-store re-store
Output	final <code>part-*.txt</code> once	final files in HDFS once	live, updated changelog topic
Setup difficulty	low (scripts, no deps)	high (cluster, HDFS, YARN)	medium (download broker, no root)
Result on same data	identical (validated)	identical	identical <i>final</i> counts

Table 12: Three paradigms on the same word-count task.

What our deliberately-simplified system lacks vs Hadoop: no HDFS (no data-locality scheduling, block replication, or rack awareness), no YARN multi-tenancy, no combiner/partitioner plugin API, no speculative execution at Hadoop’s level, no secure RPC, no job-history server. We re-implemented only the *core ideas*, map/shuffle/reduce, hash partitioning, local intermediates + remote read, re-execution, backup tasks, precisely to understand them from the inside.

15 Pain points: what broke and why

- **NFS overload.** Reading hundreds of splits from the internal NFS at once made the shared server crawl. *Fix:* intermediates to local /tmp; direct streaming of Common Crawl from Amazon, removing the NFS read path.
- **SSH fan-in / IDS.** Many parallel ssh connections per host risked tripping fail2ban. *Fix:* ControlMaster=auto multiplexing (one real TCP connection per host) plus a max_ssh cap.
- **A single dead worker froze the whole job (V1).** No failure detection meant the Main hung forever. *Fix:* the V2 fault-tolerance protocol.
- **Partial/corrupt reduce output on a mid-write crash.** *Fix:* atomic .tmp + os.replace() commit.
- **Busy worker wrongly declared dead.** An early, short lease killed healthy workers mid-MAP (the GIL starves the heartbeat thread during a 30s compute). *Fix:* raise the lease to 60s, well above one MAP, while still catching real crashes instantly via socket close.
- **Scaling wall.** We expected near-linear scaling; measurement showed a hard ceiling, a concrete lesson in Amdahl’s law.
- **Campus Wi-Fi / WSL2 flakiness.** Intermittent SSH drops; mitigated with timeouts, ServerAliveInterval, BatchMode, and idempotent deploy/clean scripts.

16 Deployment and reproducibility

16.1 Cluster deployment

```
# Deploy master + N workers, run a job on the cluster
bash src/deploy/deploy_commoncrawl.sh -w 8 -r 8 -s 20 -j wordcount
bash src/deploy/deploy_commoncrawl.sh -w 8 -r 8 -s 20 -j bigram

# Inject a fault mid-job (second terminal) and watch the master recover
bash src/deploy/fault_tolerance_demo.sh -n 1

# Validate the distributed output against a single-machine reference
python3 src/mapreduce/validate.py -i <input> -o <output> -j wordcount

# Tear everything down (idempotent; -d also deletes NFS output)
bash src/deploy/kill_commoncrawl.sh -d
```

`deploy_commoncrawl.sh` fetches live machines, SCPs the engine to NFS, downloads only the missing splits, starts the Main, then starts N workers in parallel and streams the Main log live until the job finishes.

16.2 One-command single-machine reproduction

No cluster, no teammates:

```
python3 tests/run_all.py          # correctness (4 jobs) + fault tolerance + Amdahl
    sweep
python3 tests/run_all.py --quick  # correctness + FT only
```

This spins up a real Main and several real workers on `localhost`, checks all four analyses against the reference, kills a worker to prove fault tolerance, and regenerates the Amdahl figure, exit code 0 only if everything passes.

16.3 Kafka demo

```
bash src/kafka/deploy_kafka.sh          # start a KRaft broker on /tmp
bash src/kafka/run_wordcount.sh --crawl CC-MAIN-2024-10 # stream CC -> topic -> top-N
bash src/kafka/clean_kafka.sh -a        # full teardown
```

17 Implementation map: concept to code

Every concept, optimisation and mechanism discussed in this report is backed by a specific file (and, where useful, a function) in the repository. The table below is the single index a reader can use to jump from an idea to the code that implements it.

Concept / optimisation	File : function
Live-machine discovery (school API, filter free hosts)	src/deploy/get_machines.py
CPU-load TCP server / parallel client (Day 1)	src/server/server.py, src/client/client.py
Coordinator: task queues, MAP→REDUCE barrier, detector	src/mapreduce/master.py
Worker process (MAP and REDUCE execution)	src/mapreduce/worker.py
MAP tokenise + per-job keying (4 analyses)	worker.py : _map_emit
Opt 1: combine + bytes-regex + vectorised CRC32	worker.py : _execute_map
Hash partitioning <code>crc32(k) % R</code>	worker.py : _execute_map
Opt 2: parallel SSH shuffle + ControlMaster mux	worker.py : _execute_reduce
Atomic exactly-once output (<code>.tmp</code> + <code>os.replace</code>)	worker.py : _execute_reduce
Direct S3 streaming read (<code>--direct-read</code>)	worker.py : _stream_crawl_bytes
WET/WARC header stripping	worker.py : _strip_wet_headers
Common Crawl prefetch to spill dir	src/mapreduce/download_commoncrawl.py
Failure detection (heartbeat + lease)	src/mapreduce/master.py
Re-execution + backup tasks (stragglers)	master.py : _reclaim, _pick_backup_split
Correctness validation (sequential reference)	src/mapreduce/validate.py
Scratch-disk discovery (avoid <code>tmpfs</code>)	src/benchmarks/find_scratch.sh
SSH parallelism sweep (tune <code>max_ssh</code>)	src/benchmarks/ssh_parallelism_sweep.py
Amdahl cluster sweep + figures	src/benchmarks/amdahl_bench.py, plot_amdahl.py
Deployment (1 SCP + N SSH)	src/deploy/deploy_commoncrawl.sh
Kafka Streams comparison (KRaft, no Docker)	src/kafka/run_wordcount.sh
One-command single-machine reproduction	tests/run_all.py

Table 13: From idea to implementation: where each part of the system lives.

18 Anticipated questions and design rationale (defense Q&A)

This project invites a particular kind of probing question: “why did this not break?”, “what happens at scale?”, “prove it”. Rather than leave those answers to the oral defense, we make the reasoning behind each choice explicit here, anchored in the actual code.

18.1 Memory: why we never saturated RAM, and how we would guarantee it

Q: A reducer fetches `partition_j.txt` from every map worker and sums it. Loading whole partitions into memory should blow up, why didn’t you see RAM saturation?

The answer has two halves: *why it held in practice*, and *how we would make it hold at any scale*.

Why it held: the map-side combiner. The decisive reason is that MAP *pre-aggregates* before writing (§8). In `worker.py`, the MAP phase makes one pass that folds *all* occurrences into a dict, so each `partition_j.txt` contains **one line per unique key** (the\t79745), not 79,745 separate the\t1 lines. On our real split this collapses the intermediate data from ≈ 6.6 M token emissions to ≈ 435 k distinct-key lines, more than a $10\times$ reduction, *before anything is shuffled*. Two further factors compound it:

- **Hash partitioning** `crc32(k) % R` sends each reducer only $1/R$ of the key space (with $R = 8$, ≈ 54 k keys per reducer);

- the reducer’s live state is just a `dict[str,int]` of short keys, a few megabytes, trivial against the machines’ multi-GB RAM.

So $16 \text{ splits} \times 435 \text{ k keys}$, pre-aggregated and split 8 ways, is a few MB per reducer: nowhere near saturation.

The real inefficiency: an in-memory hash-merge, not an external sort-merge. Beyond raw RAM there is an *algorithmic* objection worth naming explicitly, because it is the sharpest critique of the reduce path. For each of the M map sources the reducer calls `readlines()` and extends every line into a single `all_raw_lines` list, and only *then* folds that list into the `final_counts` dict. So at peak it holds *two* full copies of its shuffle input — the raw-line list *and* the growing hash map — and performs the merge as one big **unsorted, in-memory hash aggregation over all M partitions at once**. This departs from the canonical MapReduce reducer in three costly ways:

- **Double materialisation.** Buffering into `all_raw_lines` before aggregating is pure overhead: $O(\text{all incoming lines})$ memory plus a large short-lived list for the garbage collector to churn. Folding each line *as it streams in* would both halve peak memory and let the merge overlap the shuffle I/O.
- **No sorted runs $\Rightarrow$ no streaming k -way merge.** Because MAP writes each partition in *hash* order rather than *sorted* order, the reducer cannot do the textbook external merge of M pre-sorted runs — which touches only one line per source at a time, needs $O(\# \text{sources})$ memory, and is sequential and cache-friendly. Instead it pays for a full hash table over the entire partition, with the random-access and rehash costs that implies. This is precisely the “a ton of in-memory merging” inefficiency: the work that Hadoop/Spark push into a disk-backed sort-merge, we currently do eagerly in one RAM-resident dict.
- **Sorted output is a *partial* by-product.** We currently sort the final counts by *count* (descending) before writing, whereas an external merge sorts by *key*; the two orders differ, so the ranking would not come entirely for free. A key-sorted merge would still eliminate the separate hash table, and re-ranking the already-reduced keys by count is then a cheap final pass over the *distinct* keys only (a few $\times 10^4$ per reducer), not over the raw shuffle input.

It never bit us at 16 splits — the combiner keeps the working set at a few MB (above) — but the observation is exactly right: this is the part of the design that would not survive a TB-scale partition, and the fix is the external sort-merge listed last below.

How we guarantee it at any scale. Four levers, in order of effort:

1. **Stream the shuffle into the counter** instead of buffering it, fold each line as it arrives so memory becomes $O(\text{distinct keys in this reducer})$, not $O(\text{all lines})$:

```
for line in proc.stdout:          # iterate, do NOT readlines()
    k, _, v = line.partition("\t")
    counts[k] = counts.get(k, 0) + int(v)
```

2. **Keep the map-side combiner** (already our biggest win);
3. **Raise R** , more reducers shrink each reducer’s key space linearly;
4. **External sort-merge / spill to disk**, the real guarantee: have each MAP write its partition *sorted by key*, then the reducer does a streaming k -way merge holding only *one line per source* at a time, so memory is $O(\# \text{sources})$, independent of data size. This is exactly what Hadoop and Spark do, and it is the textbook answer to “what if even the per-reducer dict does not fit?”.

18.2 Storage and shuffle

Q: When a worker writes to its \$HOME, where does that land? On the **NFS**, not local disk, which is precisely why MAP intermediates go to /tmp (local disk), as in Figure 1 of the paper: writing intermediates to NFS would saturate the shared server (§3).

Q: /tmp can be a RAM-backed tmpfs: isn't that a trap? Yes; on some nodes /tmp is a small tmpfs that would fill under hundreds of concurrent splits. We therefore probe for a large scratch partition (find_scratch.sh) and point -local-map-dir there, falling back to /tmp only when no better disk exists.

Q: How does a reducer read intermediates written locally on the maps? By **remote read**: the Main hands the reducer the {host, map_dir} of every map; the reducer runs `ssh host cat partition_j.txt` against *all* maps **in parallel** and sums. The bytes never transit the Main (§4).

Q: Wouldn't a raw TCP transfer have been faster than ssh cat? On the wire, marginally yes; for this project, no — and the numbers say the gap is not worth chasing. A raw-TCP shuffle avoids the SSH handshake, the per-fetch crypto, and forking a remote shell+cat. But three things make it the wrong trade here:

- **We already amortised SSH's cost.** ControlMaster=auto collapses every fetch to *one* real TCP connection per host, reused across reducers (ControlPersist), and the parallel ThreadPoolExecutor floors shuffle wall-time at **1–5 s across all N** (§8.4). Shuffle is only $\approx 3\text{--}5\%$ of the wall, so even a *free* transport would shave a few percent at most — Amdahl again.
- **SSH gives us auth, encryption, and zero deployment for free.** Raw TCP means running our *own* data-serving daemon on every worker: an extra process to deploy, supervise and tear down, plus its own framing and connection management. We assume the university cluster is a *trusted* network, so the encryption/authentication SSH provides is not something we strictly need here; but on any untrusted network an unauthenticated data port would be a real exposure, and SSH gives that protection for free with no extra work on our side.
- **ssh cat is trivially correct.** A hand-rolled TCP protocol needs framing, partial-read handling, backpressure and error recovery — code that can only lose against a one-line cat at our scale.

Raw TCP becomes the right answer only when shuffle actually dominates the wall (hundreds of workers, TB-scale partitions), and even then the bigger win is *pushing* partitions from mappers as they finish so transfer overlaps compute, rather than pulling them afterwards — both are listed as future work (§8).

Q: How do you decide which node owns which key at reduce time? We assign each key with `crc32(key) % n_reducers`. It is deterministic, so all occurrences of a key land on the same reducer, **as long as the reducer count stays constant** for the job. Change it mid-job and counts shatter across reducers.

18.3 Correctness and output

Q: Are your results correct, and how do you prove it? `validate.py` recomputes each analysis on a *single* machine, sequentially, and compares key-by-key against the distributed output, and it **reuses the worker's own tokeniser** so we never compare two different logics (§12). We obtain an exact PASS.

Q: Are your writes atomic? Yes: write to a `.tmp` on the same directory, then `os.replace()` (atomic rename). A reducer that dies mid-write never leaves a half-written `part-*.txt`, and a re-executed reducer cleanly overwrites, exactly-once output (§11).

18.4 Fault tolerance

Q: How do you detect that a worker died? Heartbeats every 2s plus a lease on the Main. A killed process closes its TCP socket (immediate detection); a network partition simply lets the lease expire (§11).

Q: Why a 60s lease and not less? A real MAP spends ≈ 30 s in GIL-bound pure-Python compute during which no heartbeat can be sent; too short a lease would kill *healthy* workers. A true crash is caught instantly by the TCP close, so 60s does not slow real detection.

Q: A completed MAP on a machine that then dies: is the data lost? Yes, its partitions were on that node's local `/tmp`, so we **re-run** the map. A completed *reduce*, by contrast, is kept: its output is already on NFS, written atomically (§11).

Q: Do you handle slow machines (stragglers)? Yes, the paper's backup tasks (§3.6): past a threshold, a copy of the slow task is given to an idle worker and the first to finish wins; the late duplicate is discarded as stale.

Q: And if the Main dies? That is our stated limit: the Main is a single point of failure. Checkpoint/ recovery (§3.2 of the paper) is *designed* (sequence diagram) but *not implemented*, and we say so plainly (§15).

18.5 Performance and scaling

Q: What is the bottleneck, and can you prove it? The MAP `t_compute`, $>85\%$ of worker time, *measured* from the per-phase `WORKER_TIMING` log lines, not assumed. That is exactly what we targeted with the combiner + byte-level regex optimisation (§8, §9).

Q: Do you have a reference point for Amdahl, and the same dataset everywhere? Yes: $N = 1 \Rightarrow S = 1$, the *same dataset at every point*; we only interpolate on a single machine and *measure* from two nodes up (§10).

Q: Why does the sweep stop at 16 workers when you talk about 100 machines? Because it is a *strong-scaling* study on a fixed 16-split dataset: beyond 16 workers there is no MAP work left to hand out. “100” is the *platform capacity*; “16” is the largest worker count at which this fixed dataset still yields parallelism (full argument in §10).

18.6 Deployment and ecosystem

Q: One SCP then N SSH: why better than N SCP? `$HOME` is on NFS, *shared* by every machine: copy the engine **once** to NFS (1 SCP), then launch the servers over SSH (N SSH). Since NFS is visible everywhere, copying N times is pointless (§16).

Q: How do you find live machines? Via the school API (`tp.telecom-paris.fr`): we parse the raw JSON, filter for *live* and *free* hosts sorted by load (`get_machines.py`), and keep a verified, SSH-reachable list for the demo so we never depend on the live API mid-presentation (§2).

Q: Batch vs stream: did you actually run Kafka Streams? Yes, a Kafka broker in KRaft mode (no Docker/root), `WordCountDemo` over a real Common Crawl split. The final counts are **identical** to ours; the difference is that Kafka updates continuously via a changelog, whereas we produce one final result and stop (§13, §14).

Q: What is missing versus Hadoop? HDFS (replication, data locality), YARN (multi-tenant scheduling), a generic combiner/partitioner API, secured RPC, and a job-history server. We deliberately built the *conceptual core* to understand it, not a full Hadoop (§14).

19 Conclusion

We built a complete distributed MapReduce engine from first principles and took it all the way from a naïve proof-of-concept to a fault-tolerant, instrumented, multi-analysis system, validated against a single-machine oracle and benchmarked on real hardware. The cluster scales to $7.74\times$ on 16 nodes with a measured serial fraction of 0.065; the single-machine reproduction confirms Amdahl’s law from the other direction, hitting a $\sim 3\times$ wall under shared-resource contention. Along the way we removed the NFS bottleneck with direct Amazon streaming, hardened the protocol against worker death with heartbeats, re-execution, atomic commits and backup tasks, generalised the engine to four analyses behind one flag, and contrasted it with both Hadoop (batch) and a live Kafka Streams pipeline (stream) computing the identical result.

Limitations. Master recovery from checkpoint is designed but not implemented; the reduce-phase-source-death recovery is correct but heavy (it redoes all reduces); and large-scale runs (1000+ splits) are within the engine’s design but were exercised only into the tens of splits given lab time. These are documented frankly in the accompanying self-assessment.

References

- [1] J. Dean and S. Ghemawat. *MapReduce: Simplified Data Processing on Large Clusters*. OSDI’04, San Francisco, 2004.
- [2] Common Crawl Foundation. *WET/WARC format and crawl archives*. <https://commoncrawl.org>.
- [3] Apache Software Foundation. *Apache Kafka, KRaft mode and Kafka Streams WordCount example*. <https://kafka.apache.org>.